

作业 5 综合练习

提交要求:

1. 提交一个 word 或 pdf 文件
2. 阅读题给出运行结果，填空题给出待填空部分结果，注意标好填空的序号。
3. 程序设计题目给出完整代码，以及运行结果的截图。

一 阅读程序写结果

1. #include <iostream>
using namespace std;

```
class A {  
public:  
    A() { id = 0; cout << "A1" << endl; }  
    A(int i) { id = i; cout << "A2" << endl; }  
    A(const A& a) { id = a.id; cout << "A3" << endl; }  
    ~A() { cout << "D" << id << endl; }  
private:  
    int id;  
};
```

```
int main() {  
    A* a = new A[3]{ A(1), A(9) };  
    A* b[3] = { new A(a[1]), nullptr, new A() };  
    delete[] a;  
    delete b[0];  
    delete b[2];  
    return 0;  
}
```

2. #include<iostream>
using namespace std;

```
class A {  
private:  
    int m_a, m_b;  
public:  
    A(int i, int j) : m_a(i), m_b(j) {}  
    void Move(int x, int y) { m_a += x; m_b += y; }  
    void Show(int a) { cout << a << "<" << m_a << ", " << m_b << ">" << endl; }  
    virtual void Print() { cout << "A Print" << endl; }  
    virtual ~A() { cout << "~A" << endl; }  
};
```

```
class B : public A {  
private:  
    int m_x, m_y;  
public:  
    B(int i, int j, int k, int l) : A(i, j), m_x(l), m_y(k) {}  
    void Show(char b) { cout << b << "<" << m_x << ", " << m_y << endl; }
```

```

    void Fun() { Move(3, 5); }
    void f1() { A::Show(); }
    void Print() { cout << "B Print" << endl; }
    ~B() { cout << "~B" << endl; }
};

```

```

int main()
{
    A e(1, 2);
    e.Show(20);
    B d(3, 4, 5, 6);
    d.Fun();
    d.A::Show(20);
    d.Show(65);
    A* p = new B(10, 20, 30, 40);
    p->Print();
    delete p;
    return 0;
}

```

3. #include <iostream>
using namespace std;

```

template <typename T>
class S {
    int t;
    T e[100];
    int m;
public:
    S(int = 30);
    void P(const T &d);
    T Po();
};

```

```

template<typename T>
S<T>::S(int ms) {
    m = ms;
    t = -1;
}

```

```

template<typename T>
void S<T>::P(const T &d) {
    e[++t] = d;
}

```

```

template<typename T>
T S<T>::Po() {
    return e[t--];
}

```

```

int main() {
    int i;
    int a[5] = {1,2,3,4,5};
    S<int> s;
    for (i = 0; i < 5; i++) s.P(a[i]);
    for (i = 0; i < 5; i++) cout << s.Po();
    return 0; }

```

```
4. #include<iostream>
using namespace std;
```

```
class Counter {
public:
    Counter() : cnt(1) { cout << "C1" << endl; }
    Counter(int c) : cnt(c) { cout << "C2(" << c << ")" << endl; }
    Counter(const Counter& other) : cnt(other.cnt) { cout << "CC" << endl; }
    Counter& operator=(Counter& other) {
        cnt = other.cnt;
        cout << "Assign" << endl;
        return other;
    }
    ~Counter() { cout << "D(" << cnt << ")" << endl; }
private:
    int cnt;
};

int main() {
    Counter x, y(x), z(7);
    (x = z) = y;
    return 0;
}
```

```
5. #include<iostream>
using namespace std;
```

```
class B {
private:
    static int cnt;    // 对象个数计数
    int val;
public:
    B(int v = 0) { cnt++; val = v; }
    ~B() { cnt--; }
    friend void add(B& b) { b.val++; }
    friend void show(B b) { cout << b.val << " " << b.cnt << endl; }
};

int B::cnt = 0;

int main() {
    B b1, b2(5), b3(b2);
    add(b2);
    show(b1);
    show(b2);
    show(b3);
    return 0;
}
```

```
6. #include<iostream>
using namespace std;
```

```
class MyClass {
    char ch;
public:
    MyClass() { ch = 'A'; cout << "default cons: " << ch << endl; }
    MyClass(char c) { ch = c; cout << "cons: " << ch << endl; }
```

```

    MyClass(const MyClass& x) {
        ch = x.ch;
        cout << "copy cons: " << x.ch << endl;
    }
    MyClass& operator=(MyClass x) {
        ch = x.ch;
        cout << "op=" << 'B' << endl;
        return *this;
    }
    void set(char c) {
        ch = c;
    }
    ~MyClass() { cout << "Des: " << ch << endl; }
};

MyClass fun(MyClass& Obj) {
    Obj.set('F');
    return Obj;
}

int main() {
    MyClass obj1, obj2('D'), obj3(obj1);
    MyClass *p;
    obj3.set('H');
    p = new MyClass(obj3);
    fun(*p).set('G');
    delete p;
    obj1 = obj2;
    return 0;
}

7. class demo {
public:
    demo() { m_id = m_count++; cout << "hello" << m_id << endl; }
    ~demo() { cout << "bye" << m_id << endl; }
    static int GetCount() { return m_count; }
private:
    int m_id;
    static int m_count;
};

int demo::m_count = 0;
demo* funB();
void funA() {
    demo x, *p;
    p = funB();
    delete p;
}
demo* funB() {
    demo t, *p;
    p = new demo;
    if (demo::GetCount() > 3) cout << "right" << endl;
    return p;
}

int main() {

```

```

demo t;
funA();
cout << "Count: " << demo::GetCount() << endl;
return 0;
}

```

```

8. #include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outfile;
    outfile.open("data.txt", ios::out);
    outfile << "22 3" << "20p300s400 500.6" << endl;
    outfile.close();

    ifstream infile;
    infile.open("data.txt", ios::in);
    int num;
    while (infile >> num) {
        cout << num << endl;
        char next = infile.peek();
        if (next == 'p') cout << "pause" << endl;
        else if (next == 'a') cout << "start" << endl;
        else if (next == ' ') cout << "[S]" << endl;
    }
    // 清除流状态并回到文件开头
    infile.clear();
    infile.seekg(0, ios::beg);
    string part;
    getline(infile, part, 'p');
    cout << "[getline]部分: " << part << endl;
    infile.clear();
    infile.seekg(0, ios::beg);
    char ch;
    cout << "[get]: ";
    while (infile.get(ch)) {
        cout << ch;
    }
    infile.clear();
    infile.seekg(0, ios::beg);
    cout << "[get()] ASCII: ";
    for (int i = 0; i < 5; ++i) {
        int c = infile.get();
        if (c != EOF) cout << c << " ";
    }
    cout << endl;

    infile.close();
    return 0;
}

```

二 填空

1. 请补充代码空缺部分，使得程序能正常运行并得到如下结果：

A: X

C: 3.14

B: Y

```
#include <iostream>
using namespace std;
class A {
    char ch;
public:
    A(char c) : ch(c) {}
    void showA() const { cout << "A: " << ch << endl; }
};

class C {
    double d;
public:
    C(double v) : d(v) {}
    void showC() const { cout << "C: " << d << endl; }
};

class B : public A {
    C c;
    char ch2;
public:
    B(char a, double v, char b)
        : _____(1)_____, _____(2)_____
    {
        ch2 = b;
    }
    void display() const {
        _____(3)_____;
        c.showC();
        cout << "B: " << ch2 << endl;
    }
};

int main() {
    B obj('X', 3.14, 'Y');
    obj.display();
    return 0;
}
```

2. 以下程序定义了一个 Num 类模版。请补充代码空缺部分，使得程序能正常运行并得到预期结果（输出结果见代码注释行）。

```
// Num 类模板声明
_____(4)_____
class Num {
public:
    Num(T val);
    T getValue() const { return value; }
```

```

        void setValue(T val) { value = val; }
        T add(T other) const { return value + other; }
private:
    T value;
};
_____ (5) _____ : value(val) {}

#include <iostream>
using namespace std;
//-----
int main() {
    _____ (6) _____ n(5);           // 用于整数的 Num 对象 n
    cout << "初始值: " << n.getValue() << endl;           // 输出: 初始值: 5
    n.setValue(10);
    cout << "设置后: " << n.getValue() << endl;           // 输出: 设置后: 10
    cout << "加 3 得: " << n.add(3) << endl;           // 输出: 加 3 得: 13
    return 0;
}
//=====

```

3. 以下程序定义了一个抽象类 **Vol**，数据成员 **name** 表示名称；在此基础上，派生出 **Ball** 和 **Box** 类，函数 **vol()** 分别计算球体和长方体的体积；函数 **tVol** 计算指针数组中各对象的体积总和；主函数中定义了 4 个对象，在输出每个对象的体积后，又输出它们的体积总和。请补充空缺部分，使得程序能正常运行并得到预期结果。

```

#include <iostream>
#include <cmath>
#include <string>
using namespace std;
//-----
class Vol {
public:
    Vol(string n) : name(n) {}
    _____ (7) _____;
    virtual ~Vol() {}
    string getName() const { return name; }
private:
    string name;
};
//-----
class Ball : public Vol {
public:
    Ball(string n, double r) : Vol(n), radius(r) {}
    double vol() const { return 4.0/3.0 * 3.14159 * radius * radius * radius; }
private:
    double radius;
};
//-----
class Box : public Vol {
public:
    Box(string n, double w, double h, double d) : Vol(n), width(w), height(h), depth(d) {}
    double vol() const { return width * height * depth; }
private:
    double width, height, depth;    };

```

```
//-----
double tVol(Vol* ptr[], int n) {
    double total = 0;
    for (int i = 0; i < n; ++i)
        _____(8)_____;
    return total;
}
//-----
int main() {
    Vol* shapes[4];
    shapes[0] = new Ball("B1", 1.0);
    shapes[1] = new Box("Bx1", 2, 3, 4);
    shapes[2] = new Ball("B2", 2.0);
    shapes[3] = new Box("Bx2", 1, 1, 1);
    for (int i = 0; i < 4; ++i)
        cout << shapes[i]->getName() << ":" << shapes[i]->vol() << " ";
    cout << endl;
    cout << "Total volume: " << _____(9)_____ << endl;
    for (int i = 0; i < 4; ++i)
        _____(10)_____;
    return 0;
}
```

程序输出: **B1:4.18879 Bx1:24 B2:33.5103 Bx2:1**
Total volume: 62.6991

三 程序设计

1. 设计一个账单基类 **Bill** 和派生类 **Disc**，并实现从文本文件读取多行数据、处理支付、将结果写入文件的功能。基类的实现代码如下：

```
class Bill {
private:
    string acc; //账户号 acc
    double bal; //余额 bal
public:
    Bill(string a = "0000", double b = 0) : acc(a), bal(b) {}
    virtual void show() const {
        cout << "Account: " << acc << ", Balance: " << bal << endl;
    }
    string getAcc() const { return acc; }
    double getB() const { return bal; }
    void setB(double b) { bal = b; }
    virtual ~Bill() {}
};
```

派生类 **Disc** 公有继承 **Bill**，要求：

- 新增私有数据成员：折扣率 **dis**（**double** 类型，0~1 之间）。
- 构造函数：初始化 **acc**、**bal** 和 **dis**。
- 重写 **show()**：显示账户号、余额和折扣率。

- 成员函数 `bool pay(double amt)`: 处理支付操作。若支付金额 `amt` 大于等于 实际应付金额 (即 `bal * (1 - dis)`) , 则支付成功, 更新余额 `bal = bal - amt` (可能为负), 返回 `true`; 否则返回 `false`。

主函数要求:

- 新建并将下列信息写入文本文件 `"in.txt"`。
文件内容:
0000 0 0.0
123456 1000.0 0.2
999999 500.0 0.1
- 读取文件, 每行三个数字分别代表 账户号、余额和折扣率。每一行数据创建对应的 `Disc` 对象, 调用函数将信息输出显示。
- 调用 `pay`, 支付金额固定为 当前余额的一半, 根据返回值输出 `"success!"` 或 `"failed!"`。

2. 设有一个类 `Stack`, 实现了简单的栈(用动态数组实现), 栈是仅在数组一端操作的数据结构。栈操作的一端是数组尾端, 栈中数据按后进先出的顺序进行入栈, 出栈等操作。栈类 `Stack` 的声明和部分函数定义如下所示, 请补充完成剩余函数定义。

```
#include<iostream>
#include<cstdlib>//exit(1) 终止函数
using namespace std;
class Stack{
    int myCapacity;//栈容量
    int myTop;//栈顶位置
    int*myArray;
public:
    Stack(int numElements=128);//补充完成构造函数 3分
    Stack(const Stack& original);//复制构造函数
    ~Stack(){delete[] myArray;}//析构函数
    const Stack& operator=(const Stack& rightHandSide);//补充完成赋值函数, 4分
    bool empty() const {return (myTop==0);} //栈判空函数
    void push(const int& value);//补充完成入栈函数, 3分
    int top() const;//获取栈顶元素函数
    void pop();//补充完整出栈函数, 3分
};

//=====Definition of Stack copy constructor=====
Stack::Stack(const Stack& original): myCapacity(original.myCapacity),
myTop(original.myTop){
//Get new array for copy
myArrav = new int[myCapacity];
if(myArray!=0){
    for( int pos=0;pos<=myTop: pos++){
        myArray[pos]=original.myArray[pos];
    }
}
else{
    cout<<"Inadequate memory to allocate stack\n";
    exit(1);
}
}

//=====Definition of top()=====
int Stack::top() const{
    if( !empty()) return(myArray[myTop]);
}
```

```

        else{
            cout<<"Stack is empty\n";
            exit(1);
        }
    }
}

```

3. 现有一个字符序列，采用单向链表存储。链表结点类为 **node**，链表类为 **List**，完成 **List** 类的成员函数。

类定义及主函数框架如下：

```

#include<iostream>
using namespace std;
//-----
class node {
public:
    node(char d = 0, node* n = nullptr) : data(d), next(n) {}
    char data;
    node* next;
};
//-----
class List {
    node* head;
public:
    List() : head(nullptr) {}
    ~List();           //析构函数，释放内存
    void Create(char *array, int n);    // 根据字符数组创建链表
    void RemoveDup();                  // 删除重复字符
    void ListRev();                    // 逆置链表
    void Display();                    // 输出链表
    int Stat() const;                  // 统计元素个数
};
//-----
int main() {
    List list;
    char arr[] = {'A','B','C','B','A','D','B','E','C'};
    list.Create(arr, 9);
    cout << "Original: ";
    list.Display();

    list.RemoveDup();
    cout << "Remove duplicates, there are "<<list.Stat() << " nums: ";
    list.Display();

    list.ListRev();
    cout<<"Reverse nums: ";
    list.Display();
    return 0;
}

```

主函数的输出如下：

Original: A,B,C,B,A,D,B,E,C

Remove duplicates, there are 5 nums: A,B,C,D,E

Reverse nums: E,D,C,B,A